CLI Chess Engine using Minimax Algorithm

Submitted by:

<u>Student Name</u>: Divyansh Agarwal

<u>Reg No</u>: 25BCE11367

<u>Branch</u>: CSE CORE

<u>Course</u>: Fundamentals in AI and ML CSA2001

<u>Course Code</u>: CSA2001

INTRODUCTION

Chess is a classic strategy-based board game that has long served as a benchmark for artificial intelligence research. The complexity of possible moves and the need for strategic planning make it an ideal problem for studying decision-making algorithms. With the advancement of computing, chess engines have evolved from simple rule-based systems to highly sophisticated AI-driven models.

This project presents the development of a **Command Line Interface (CLI) based Chess Engine** using Python. The system allows a human player to compete against an AI opponent that makes intelligent decisions using the **Minimax algorithm**, enhanced with **Alpha-Beta pruning** and **Quiescence search**. These techniques enable the engine to efficiently evaluate possible game states and select optimal moves within a reasonable time.

Unlike graphical chess applications, this project focuses entirely on a terminal-based interaction, demonstrating that powerful AI systems can be built even without a graphical interface. The CLI design ensures simplicity, portability, and ease of execution across different platforms.

The primary aim of this project is to bridge theoretical concepts of artificial intelligence with practical implementation. By developing this chess engine, key concepts such as recursive search, heuristic evaluation, and optimization techniques are explored in depth, providing a strong foundation in AI and algorithm design.

MOTIVATION OF THE PROJECT

The motivation behind this project stems from the desire to understand how artificial intelligence can be applied to solve complex, strategic problems. Chess, being a game of logic, planning, and foresight, serves as an ideal platform to explore AI concepts such as decision-making, search algorithms, and optimization techniques.

In modern computing, AI is widely used in various domains, but understanding its core principles requires hands-on implementation. This project provides an opportunity to implement and analyse fundamental algorithms like the **Minimax algorithm** and **Alpha-Beta pruning**, which form the backbone of many intelligent systems.

Additionally, developing a CLI-based chess engine helps in strengthening programming skills, especially in problem-solving, recursion, and data handling. It also demonstrates how efficient solutions can be built without relying on graphical interfaces, focusing purely on logic and algorithmic efficiency.

Overall, the project is motivated by the goal of combining theoretical knowledge of artificial intelligence with practical implementation, thereby gaining a deeper understanding of how machines can simulate intelligent behaviour in real-world scenarios.

PROBLEM STATEMENT

The objective of this project is to design and implement a **Command Line Interface (CLI) based chess engine** that can intelligently compete against a human player. The system must be capable of generating legal moves, evaluating different board positions, and selecting the most optimal move using artificial intelligence techniques.

The key challenge lies in handling the vast number of possible move combinations in chess and making efficient decisions within a limited time. Therefore, the problem focuses on implementing optimized search algorithms such as **Minimax with Alpha-Beta pruning** to reduce computational complexity while maintaining accuracy in decision-making.

Additionally, the system should provide a smooth and interactive user experience through the terminal, ensuring correct input handling, game state management, and rule enforcement.

OBJECTIVE OF THE PROJECT

The main objectives of this project are:

- To develop a fully functional **CLI-based chess application** that allows a human player to play against an AI opponent.
- To implement the **Minimax algorithm** for decision-making in a two-player game environment.
- To optimize the search process using **Alpha-Beta pruning**, thereby improving performance and efficiency.
- To incorporate **Quiescence search** for better evaluation of tactical positions and to reduce the horizon effect.
- To design a clean and user-friendly terminal interface that supports move input, undo functionality, and game restart.
- To gain practical experience in applying artificial intelligence concepts such as search algorithms, recursion, and heuristic evaluation.

.

EXISTING METHODS

Various approaches have been used historically to develop chess-playing systems. Early methods relied on **brute-force search**, where all possible moves were explored without

optimization. While simple to implement, these methods were computationally expensive and inefficient.

Modern chess engines use more advanced techniques such as **Minimax-based search algorithms**, combined with optimizations like **Alpha-Beta pruning** to reduce unnecessary computations. Additionally, many engines incorporate **heuristic evaluation functions** to estimate the strength of a board position based on factors like material balance and piece positioning.

Advanced systems further enhance performance using **opening books**, which store predefined strong opening moves, and **transposition tables**, which cache previously evaluated positions to avoid redundant calculations. In recent years, **machine learning and neural networks** have also been introduced, enabling engines to learn patterns and improve decision-making over time.

PROS AND CONS OF THE STATED METHODS

Advantages:

- **Efficient Decision Making:** Algorithms like Minimax with Alpha-Beta pruning significantly reduce the number of positions evaluated.
- **Improved Performance:** Optimization techniques allow deeper search within limited time.
- **Strategic Gameplay:** Heuristic evaluation enables the engine to make intelligent and competitive moves.
- **Scalability:** Advanced methods like transposition tables and opening books can further enhance performance.

Disadvantages:

- **High Computational Cost:** Even with optimizations, deeper searches require significant processing power.
- **Limited Depth:** The engine cannot explore all possibilities due to time and hardware constraints.
- **Heuristic Limitations:** Evaluation functions may not capture all positional complexities accurately.
- **Complex Implementation:** Advanced techniques like neural networks and caching mechanisms increase system complexity.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware: Standard PC
Software: Python 3.x, python-chess library, VS Code

METHODOLOGY AND GOAL

The methodology of this project is based on implementing a structured approach to simulate intelligent decision-making in a chess game using artificial intelligence techniques.

The system begins by initializing a standard chessboard using the python-chess library, which ensures correct rule enforcement and move validation. The user interacts with the system through a Command Line Interface (CLI), entering moves in Standard Algebraic Notation (SAN).

Once a valid move is entered, the AI engine processes the current board state using the **Minimax algorithm**, which recursively explores possible future moves. To improve efficiency, **Alpha-Beta pruning** is applied to eliminate branches of the search tree that do not influence the final decision. Additionally, **Quiescence search** is used to extend evaluation in unstable positions, particularly during captures, ensuring more accurate decision-making.

The goal of this methodology is to balance **performance and accuracy**, allowing the engine to make intelligent moves within a reasonable time while maintaining a smooth user experience in a terminal-based environment.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

The system is divided into several functional modules to ensure modularity, clarity, and efficient execution:

1. Input Module

This module handles user interaction. It accepts moves entered in SAN format and validates them using the chess library. It also processes special commands like undo, restart, and exit.

2. Board Management Module

This module maintains the current state of the chessboard. It ensures that all moves follow official chess rules and updates the board after each move.

3. AI Engine Module

This is the core component of the system. It uses the Minimax algorithm along with Alpha-Beta pruning to evaluate possible moves and select the best one. It also integrates Quiescence search for better handling of tactical positions.

4. Evaluation Module

This module assigns a numerical score to a given board position based on material value and basic positional understanding. It helps the AI determine which positions are favorable.

5. Output Module

This module is responsible for displaying the chessboard in the terminal using Unicode characters and ANSI colors. It also provides feedback such as engine moves, game status, and results.

ALGORITHM DEVELOPMENT

The core of this project is based on the **Minimax algorithm**, which is widely used in two-player strategy games like chess. The algorithm simulates all possible moves up to a certain depth and assumes that both players play optimally.

In this implementation, the AI acts as the **maximizing player**, trying to maximize its score, while the human player acts as the **minimizing player**, trying to reduce the AI's advantage. The algorithm recursively evaluates all possible game states and selects the move that leads to the best possible outcome.

To improve efficiency, **Alpha-Beta pruning** is integrated into the Minimax algorithm. This optimization eliminates branches of the search tree that do not need to be explored because they cannot influence the final decision. This significantly reduces the number of nodes evaluated and allows deeper searches within the same time constraints.

Additionally, the system implements **Quiescence search**, which extends the evaluation beyond the fixed depth limit in unstable positions (such as captures). This helps in avoiding the *horizon effect*, where the engine might miss critical tactical moves due to shallow search depth.

Overall, the algorithm ensures a balance between computational efficiency and decision accuracy, enabling the engine to play competitive chess within limited resources.

SOFTWARE ARCHITECTURAL DIAGRAM

CLI Interface → Game Controller → Chess Engine → Evaluation Function

Software Architecture Diagram

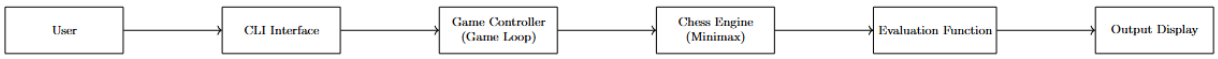


Figure 1: Software Architecture of CLI Chess Engine

CODING

The project is implemented in Python using modular functions for evaluation, search, and UI. The implementation of the CLI Chess Engine is carried out using the **Python programming language**, leveraging its simplicity and strong support for algorithm development. The project uses the **python-chess library** to handle core chess functionalities such as board representation, legal move generation, and rule enforcement.

The code is structured in a modular manner to improve readability, maintainability, and scalability. The main components of the implementation include:

- **Board Initialization:**
  The chessboard is initialized using the chess.Board() class, which maintains the current game state and ensures that all moves follow official chess rules.

- **User Input Handling:**
  The system accepts moves in **Standard Algebraic Notation (SAN)** through the terminal. Input validation is performed using built-in parsing functions to ensure only legal moves are executed.

- **Game Loop:**
  A continuous loop manages the flow of the game, alternating between user input and AI-generated moves until the game reaches a terminal state such as checkmate, stalemate, or draw.

- **AI Engine Implementation:**
  The Minimax algorithm is implemented recursively to explore possible game states. The algorithm evaluates each move up to a specified depth and selects the optimal move based on maximizing or minimizing the evaluation score.

- **Optimization Techniques:**
  Alpha-Beta pruning is integrated into the Minimax algorithm to reduce the number of nodes evaluated, thereby improving efficiency. Additionally, Quiescence search is used to extend evaluation in tactical positions.

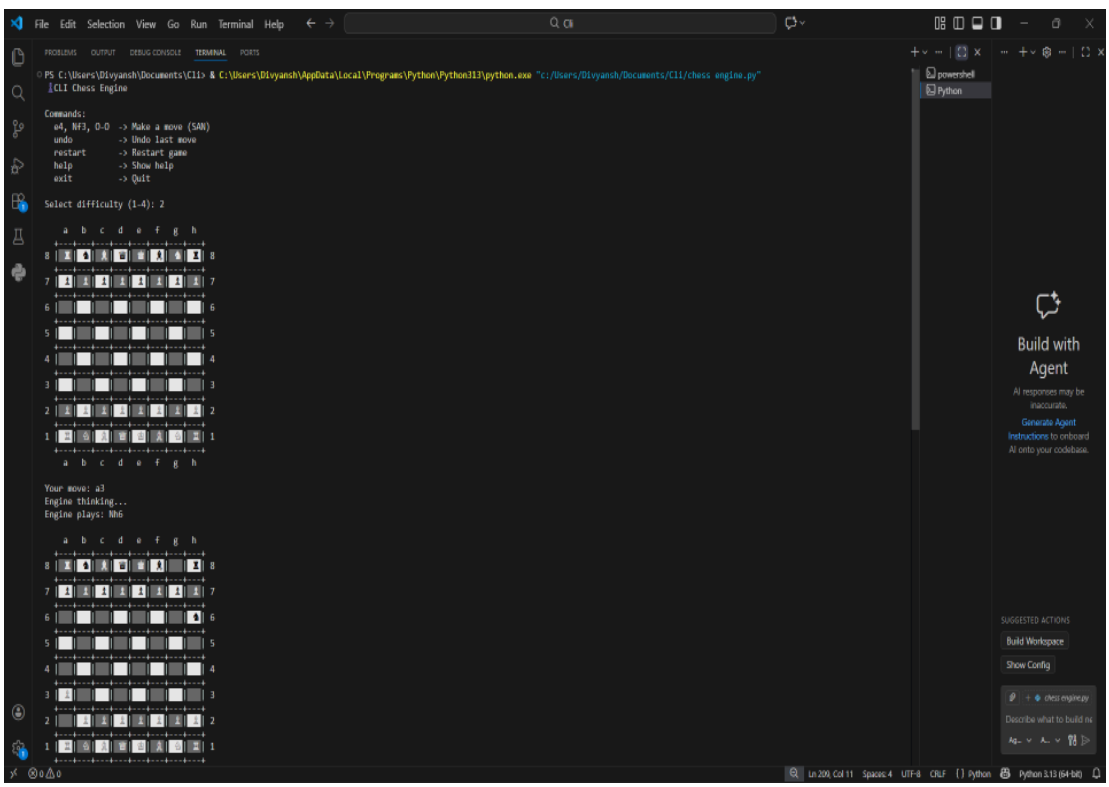
- **Evaluation Function:**
  The board is evaluated based on material balance, assigning numerical values to different pieces. This helps the AI determine favourable positions.

- **Output Rendering:**
  The chessboard is displayed in the terminal using **Unicode characters and ANSI colour codes**, providing a visually clear and user-friendly interface.

Overall, the coding implementation ensures a balance between **algorithmic efficiency and user interaction**, resulting in a functional and interactive CLI-based chess system.

OUTPUT

The output is a colored chessboard displayed in terminal with interactive gameplay.



KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

Efficient move generation, evaluation scoring, and user interaction through CLI.The system is designed with a focus on combining artificial intelligence techniques with an efficient command-line interface. The key implementations of the system are outlined as follows:

- **Efficient Board Representation:**
  The chessboard is implemented using the python-chess library, which provides a reliable and rule-compliant representation of the game state. This ensures accurate move validation and simplifies complex rule handling.

- **Minimax-Based Decision Making:**
  The core intelligence of the system is built using the Minimax algorithm. It recursively evaluates possible future moves and selects the optimal move based on maximizing the AI's advantage while minimizing the opponent's gain.

- **Alpha-Beta Pruning Optimization:**
  To improve performance, Alpha-Beta pruning is integrated into the Minimax algorithm. This reduces the number of unnecessary nodes explored, enabling deeper search within limited computational time.

- **Quiescence Search Implementation:**
  The system extends evaluation in unstable positions, such as capture sequences, using Quiescence search. This helps avoid inaccurate evaluations caused by the horizon effect.

- **Heuristic Evaluation Function:**
  A scoring mechanism is implemented based on material values assigned to each chess piece. This allows the engine to quantitatively assess board positions and make informed decisions.

- **Interactive CLI Interface:**
  The system provides a user-friendly command-line interface where users can input moves using Standard Algebraic Notation. Additional commands such as undo, restart, and exit enhance usability.

- **Dynamic Game Loop Control:**
  A continuous game loop manages turn-based gameplay, ensuring proper synchronization between user input and AI responses until the game reaches a conclusion.

- **Visual Board Rendering:**
  The chessboard is displayed using Unicode characters and ANSI color codes, improving readability and enhancing the user experience within a terminal environment.

SIGNIFICANT PROJECT OUTCOMES

Developed a working AI-based chess engine capable of playing legal and competitive moves.

The development of the CLI Chess Engine resulted in several important outcomes that demonstrate both technical implementation and conceptual understanding of artificial intelligence:

- **Successful Implementation of AI-Based Chess Engine:**
  A fully functional chess engine was developed that can compete against a human player using intelligent decision-making algorithms.

- **Application of Game Theory Concepts:**
  The project effectively demonstrates the practical application of game theory principles, particularly through the use of the Minimax algorithm in a two-player adversarial environment.

- **Efficient Search Optimization:**
  By incorporating Alpha-Beta pruning, the system significantly reduces computational overhead, allowing deeper search levels and improved performance.

- **Improved Decision Accuracy:**
  The integration of Quiescence search enhances the engine's ability to evaluate complex tactical positions, reducing errors caused by shallow search depth.

- **Interactive CLI-Based System:**
  The project successfully delivers a user-friendly terminal interface, enabling smooth interaction, move input, and real-time gameplay without requiring a graphical interface.

- **Robust Game Management:**
  The system correctly handles all game states, including check, checkmate, stalemate, and draw conditions, ensuring rule-compliant gameplay.

- **Enhanced Programming and AI Skills:**
  The project contributed to a deeper understanding of recursion, algorithm optimization, and heuristic evaluation, strengthening both programming and problem-solving skills.

TESTING AND REFINEMENT

Tested using various move scenarios and improved stability and error handling. The developed CLI Chess Engine was thoroughly tested to ensure correctness, stability, and efficiency of the system. Various test cases and gameplay scenarios were considered to validate both the functional and algorithmic aspects of the application.

- **Functional Testing:**
  The system was tested for correct execution of chess rules, including valid move

generation, illegal move rejection, and proper handling of special moves such as castling and captures. The input system was verified to ensure that only valid Standard Algebraic Notation (SAN) moves are accepted.

- **AI Behavior Testing:**
  The Minimax algorithm was tested across different depth levels to evaluate the quality of moves generated by the engine. It was observed that higher depth values improved decision-making at the cost of increased computation time.

- **Edge Case Testing:**
  Special game scenarios such as check, checkmate, stalemate, and draw conditions were tested to ensure accurate detection and proper game termination.

- **Performance Testing:**
  The efficiency of the engine was analyzed by observing response times for different difficulty levels. The inclusion of Alpha-Beta pruning significantly reduced unnecessary computations and improved performance.

- **Error Handling and Debugging:**
  Several runtime errors, such as invalid user inputs and incorrect move parsing, were identified and resolved during testing. Improvements were made to enhance system robustness and user experience.

- **User Experience Refinement:**
  The CLI interface was refined to improve readability and usability. Features such as colored board display, clear prompts, and additional commands (undo, restart, help) were added to enhance interaction.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- **Game Development:**
  The project provides a foundational framework for developing intelligent game systems. The techniques used can be extended to create more advanced games with AI-driven opponents.

- **Artificial Intelligence Research and Education:**
  This system serves as an effective learning tool for understanding core AI concepts such as search algorithms, optimization techniques, and heuristic evaluation. It can be used in academic environments to demonstrate practical AI implementation.

- **Decision-Making Systems:**
  The Minimax algorithm and its optimizations are applicable in real-world decision-making systems where outcomes depend on multiple competing factors, such as strategic planning and resource allocation.

- **Optimization Problems:**
  Techniques like Alpha-Beta pruning can be applied to reduce computational complexity in various optimization problems, including scheduling, logistics, and operations research.

- **Simulation and Training Systems:**
  Similar AI models can be used in simulation environments for training purposes, where systems need to respond intelligently to user actions.

- **Foundation for Advanced AI Systems:**
  The project provides a base for further enhancements using machine learning and neural networks, which are widely used in modern AI applications such as autonomous systems and predictive modeling.

CONTRIBUTION/ FINDINGS OF THE PROJECT

Demonstrated how AI algorithms can be applied to strategic games. This project contributes to the practical understanding and implementation of artificial intelligence techniques in a real-world problem domain. The key findings and contributions of the project are as follows:

- **Practical Implementation of AI Algorithms:**
  The project successfully demonstrates how theoretical concepts such as the Minimax algorithm and Alpha-Beta pruning can be applied to develop an intelligent decision-making system.

- **Understanding of Search Optimization:**
  It was observed that Alpha-Beta pruning significantly reduces the number of nodes evaluated, thereby improving performance without affecting the accuracy of the result.

- **Impact of Search Depth on Performance:**
  Increasing the search depth improves the quality of moves generated by the engine but also increases computation time, highlighting the trade-off between accuracy and efficiency.

- **Importance of Evaluation Functions:**
  The effectiveness of the AI depends heavily on the evaluation function. Even a simple material-based evaluation can produce reasonable gameplay, but more advanced heuristics can further enhance performance.

- **Handling of Tactical Positions:**
  The use of Quiescence search helps in avoiding poor decisions in unstable positions, emphasizing the importance of extending search in critical scenarios.

- **CLI-Based System Efficiency:**
  The project demonstrates that a fully functional and interactive AI system can be implemented efficiently using a command-line interface without the need for complex graphical components.

- **Enhanced Problem-Solving Skills:**
  The development process improved understanding of recursion, algorithm design, debugging, and system structuring, contributing to overall technical growth.

LIMITATION/CONSTRAINTS OF THE SYSTEM

Limited depth due to computational constraints. No opening database. Despite successfully implementing a functional CLI-based chess engine, the system has certain limitations that restrict its performance and scalability:

- **Limited Search Depth:**
  The engine operates within a fixed depth due to computational constraints. Increasing the depth significantly improves move quality but also increases execution time, making real-time performance challenging.

- **Basic Evaluation Function:**
  The current evaluation is primarily based on material balance and lacks advanced positional understanding such as pawn structure, mobility, and king safety, which limits the strategic strength of the engine.

- **No Opening or Endgame Database:**
  The system does not utilize opening books or endgame tablebases. As a result, it may not play optimally in well-known opening sequences or simplified endgame scenarios.

- **High Computational Cost:**
  Even with Alpha-Beta pruning, the number of possible game states grows exponentially, leading to increased processing time at higher depths.

- **Lack of Parallel Processing:**
  The engine runs on a single thread and does not take advantage of multi-core processing, which could otherwise improve performance.

- **CLI-Only Interface:**
  While efficient, the command-line interface may not be as intuitive or visually appealing as graphical user interfaces, limiting accessibility for some users.

- **No Learning Capability:**
  The system does not incorporate machine learning techniques, meaning it cannot learn from past games or improve over time.

CONCLUSIONS

CLI Chess Engine developed in this project successfully demonstrates the practical implementation of artificial intelligence techniques in a strategic game environment. By utilizing the **Minimax algorithm**, along with **Alpha-Beta pruning** and **Quiescence search**, the system is capable of making intelligent and optimized decisions while maintaining computational efficiency.

The project effectively bridges the gap between theoretical concepts of AI and their real-world application. It showcases how complex decision-making problems can be solved using structured algorithms and heuristic evaluation methods. Additionally, the CLI-based design proves that powerful and interactive systems can be built without relying on graphical interfaces.

Through this project, significant insights were gained into algorithm design, recursion, optimization, and system development. Overall, the implementation meets the objectives of creating a functional, efficient, and interactive chess engine, while also providing a strong foundation for further advancements in artificial intelligence.

FUTURE ENHANCEMENTS

Add GUI, improve evaluation, use opening books and transposition tables. While the current system is functional and efficient, several enhancements can be implemented to improve its performance, usability, and intelligence:

- **Advanced Evaluation Function:**
  Incorporate positional factors such as pawn structure, mobility, king safety, and piece coordination to improve decision-making quality.

- **Opening Book Integration:**
  Add a database of common opening moves to enable faster and more accurate gameplay in the initial stages.

- **Transposition Tables:**
  Implement caching techniques to store previously evaluated positions, reducing redundant computations and improving efficiency.

- **Graphical User Interface (GUI):**
  Develop a user-friendly graphical interface to enhance visual interaction and accessibility.

- **Parallel Processing:**
  Utilize multi-threading or parallel computing to improve performance and enable deeper search levels.

- **Machine Learning Integration:**
  Incorporate learning algorithms or neural networks to allow the engine to improve over time based on experience.

- **Move Highlighting and Analysis Features:**
  Add features such as move suggestions, evaluation scores, and game analysis to enhance user engagement.

REFERENCES

1. Python-Chess Documentation
2. AI textbooks on Minimax
3. Online resources